

UNIVERSIDADE FEDERAL DO PARANÁ

DANIELA TAMY YUKI - GRR20241665

LAURA KLEMBA CORDEIRO - GRR GRR20243748

NATHALIA LYRA VARELA DE ALBUQUERQUE – GRR20245377

LEONARDO DE LIMA PEREIRA – GRR20214905

Diário de Bordo: Simulador de Caixa de Supermercado

CURITIBA

2025

ESTRUTURA DE DADOS I - DS130

Diário de Bordo: Simulador de Caixa de Supermercado

Tarefa apresentada ao curso Superior Técnico em Análise e Desenvolvimento de Sistemas do Setor de Educação Profissional e Tecnológica da Universidade Federal do Paraná (UFPR), como requisito parcial à obtenção do título de Técnico em Análise e Desenvolvimento de Sistemas.

Professora. Helcio Padilha

CURITIBA

2025

SUMÁRIO

1. Introdução.....	4
2. Módulo cliente.....	5
2.1. cliente.h.....	5
2.2. cliente.c.....	6
2.3. Dificuldades.....	7
3. Módulo fila.....	7
3.1. fila.h.....	7
3.2. fila.c.....	8
3.3. Dificuldades.....	9
4. Módulo atendimento.....	9
4.1. atendimento.h.....	9
4.2. atendimento.c.....	10
4.3. Dificuldades.....	10
5. Main.....	10
5.1. main.c.....	10
5.2. Dificuldades.....	12
6. Conclusão.....	12

1. Introdução

O projeto “Sistema de Atendimento em Caixas de Supermercado” foi desenvolvido em linguagem C e tem como objetivo simular o funcionamento de filas comuns e preferenciais em um supermercado, aplicando conceitos básicos de Estrutura de Dados.

O sistema foi dividido em módulos, de forma a organizar melhor o código e separar as responsabilidades de cada parte:

- **Cliente (cliente.c / cliente.h):** responsável pelo cadastro dos clientes, armazenando informações como nome, tipo (comum ou preferencial) e quantidade de itens.
- **Fila (fila.c / fila.h):** responsável pela implementação das filas utilizadas no sistema, tanto a fila comum quanto a fila preferencial.
- **Atendimento (atendimento.c / atendimento.h):** controla a lógica de atendimento dos clientes, incluindo a alternância entre as filas, o histórico de atendimentos e o cálculo das estatísticas.
- **Menu (main.c):** fornece a interface de interação com o usuário, permitindo executar as funcionalidades do sistema.

Para o gerenciamento dos clientes, foi utilizada a estrutura de fila, pois ela representa de forma natural o funcionamento de um caixa de supermercado, onde o primeiro cliente a entrar é o primeiro a ser atendido (FIFO – *First In, First Out*).

O sistema utiliza duas filas, uma comum e outra preferencial, ambas com alocação dinâmica de memória. Isso significa que não há um tamanho fixo para as filas, permitindo a entrada de novos clientes conforme a necessidade, apenas criando um novo nó na memória.

A lógica de atendimento segue uma regra de alternância, na qual dois clientes da fila comum são atendidos para cada cliente da fila preferencial, buscando manter um equilíbrio entre prioridade e fluidez no atendimento.

Por fim, o sistema registra informações como tempo estimado e quantidade de itens atendidos, possibilitando a geração de estatísticas finais e reforçando a aplicação prática dos conceitos estudados em Estrutura de Dados.

Vídeo YT apresentação trabalho: <https://www.youtube.com/watch?v=s0KYqLz1UeA>

Responsabilidades do grupo:

Integrante	Responsabilidade	Arquivo
Daniela Tamy	Módulo cliente	cliente.h / cliente.c
Laura Klemba	Main	main.c
Leonardo Pereira	Módulo atendimento	atendimento.h / atendimento.c
Nathalia Lyra	Módulo fila	fila.h / fila.c

2. Módulo cliente

2.1. cliente.h

Define a estrutura do cliente, que armazena as informações para o atendimento: nome, tipo, itens e tempo de atendimento. Além disso, declara as funções do módulo.

Estruturas:

```
typedef struct Cliente {  
  
    char nome[50];  
  
    int tipo;           // 0 = comum, 1 = preferencial  
  
    int itens;  
  
    int tempoAtendimento; // calculado depois  
  
} Cliente;
```

Funções Principais:

```
// Cria um cliente dinamicamente  
  
Cliente criarCliente(char nome[], int tipo, int itens);  
  
// Libera memória do cliente  
  
void liberarCliente(Cliente* c);
```

```
// Exibe dados do cliente

void imprimirCliente(Cliente* c);

// função para calcular tempo de atendimento - auxiliar

int calcularTempoAtendimento(int itens);

// função para verificar se é preferencial - auxiliar

int isPreferencial(int tipo);
```

2.2. cliente.c

A função `criarCliente()` é responsável por iniciar um cliente com os dados necessários. A função recebe o nome, o tipo e a quantidade de itens, copia essas informações para a estrutura e calcula automaticamente o tempo estimado de atendimento por meio da função auxiliar `calcularTempoAtendimento()`.

```
Cliente criarCliente(char nome[], int tipo, int itens) {

    Cliente novo;

    strcpy(novo.nome, nome);

    novo.tipo = tipo;

    novo.itens = itens;

    novo.tempoAtendimento = calcularTempoAtendimento(itens);

    return novo;

}
```

A função `liberarCliente()` é responsável por liberar a memória de um cliente alocado dinamicamente.

```
void liberarCliente(Cliente* c) {

    if (c) {

        free(c);

    }

}
```

A função `imprimirCliente()` é responsável por exibir todas as informações de um cliente na tela, como nome, tipo, quantidade de itens e tempo estimado de atendimento.

```
void imprimirCliente(Cliente* c) {  
  
    if (!c) return;  
  
    printf("Nome: %s\n", c->nome);  
    printf("Tipo: %s\n", c->tipo == 1 ? "Preferencial" : "Comum");  
    printf("Itens: %d\n", c->itens);  
    printf("Tempo Estimado: %d segundos\n", c->tempoAtendimento);  
}
```

A função `calcularTempoAtendimento()` é responsável por calcular o tempo total de atendimento do cliente.

```
int calcularTempoAtendimento(int itens) {  
  
    // Exemplo: cada item leva 2 segundos para ser processado + 1  
    minuto para pagamento  
  
    return itens * 2 + 60;  
}
```

A função `isPreferencial()` é responsável por verificar se o cliente pertence à fila preferencial.

```
int isPreferencial(int tipo) {  
  
    return tipo == 1;  
}
```

2.3. Dificuldades

As dificuldades encontradas neste módulo foram planejar as funções auxiliares, que precisaram ser revisadas após a construção dos outros módulos.

3. Módulo fila

3.1. fila.h

Define a estrutura das filas do supermercado e declara suas funções.

Estruturas:

```
#ifndef FILA_H
#define FILA_H

#include "cliente.h"

typedef struct NoFila {
    Cliente cliente;
    struct NoFila* proximo;
} NoFila;

typedef struct {
    NoFila* inicio;
    NoFila* fim;
    int tamanho;
} Fila;
```

O struct noFila representa cada elemento da fila e o struct Fila consiste na estrutura principal da fila.

Funções Principais:

```
void inicializarFila(Fila* f);
int filaVazia(Fila* f);
void enfileirar(Fila* f, Cliente c);
Cliente desenfileirar(Fila* f);
void exibirFila(Fila* f, const char* tipoFila);
void liberarFila(Fila* f);
```

São seis as funções desse módulo, tendo como objetivo realizar as operações básicas da estrutura, como inicializar e liberar fila.

3.2. fila.c

- void inicializarFila(Fila* f);
 - Prepara uma fila vazia para uso, declarando o início e fim como null e tamanho como 0.
- int filaVazia(Fila* f);
 - Verifica se a fila está vazia com base no início. Se o início estiver null então a fila está vazia.
- void enfileirar(Fila* f, Cliente c);
 - Adiciona um cliente ao final da fila.
 - Primeiramente, cria um novo nó e preenche os dados do cliente.
 - Em seguida, insere na fila. Caso seja o primeiro elemento insere no início, caso contrário, insere como seguinte do antigo nó fim.
 - Por fim, atualiza o fim para o elemento inserido e incrementa o tamanho da fila.
- Cliente desenfileirar(Fila* f);

- Remove o primeiro nó inserido, ou seja, o início da fila.
- Primeiramente verifica se a fila está vazia. Caso esteja, envia mensagem de aviso.
- Guarda referência e salva os dados do primeiro nó em um nó temporário.
- Redireciona o ponteiro de início para o nó seguinte. Caso não haja mais nenhum elemento na fila, atualiza o fim para null.
- Libera a memória do nó temporário.
- Decrementa o tamanho da fila.
- Retorna as informações removidas.
- void `exibirFila(Fila* f, const char* tipoFila);`
 - Exibe as informações de nome e quantidade de itens de cada cliente segundo tipo de fila, comum ou preferencial.
- void `liberarFila(Fila* f);`
 - Limpa toda a fila e libera a memória através da estrutura while.

3.3. Dificuldades

As maiores dificuldades foram elaborar as funções enfileirar e desenfileirar visto a necessidade de gerenciamento de memória, manipulação de ponteiros e gerenciamento de exceções (casos de fila vazia ou tratamento de null), para além da necessidade de manipulação dos conteúdos presentes no módulo cliente.

4. Módulo atendimento

4.1. atendimento.h

Estruturas:

```
#ifndef ATENDIMENTO_H
#define ATENDIMENTO_H
#include "fila.h"

typedef struct {
    int total_atendidos_comum;
    int total_atendidos_preferencial;
    int total_itens_comum;
    int total_itens_preferencial;
    int tempo_total_comum;
    int tempo_total_preferencial;
    Fila historico_atendimentos;
    int contador_atendimento_comum;
} Atendimento;
```

Funções Principais:

```
Atendimento* inicializarAtendimento();  
void liberarAtendimento(Atendimento* a);  
void atenderProximoCliente(Fila* fComum, Fila* fPreferencial,  
Atendimento* a);  
void exibirHistoricoAtendimentos(Atendimento* a);  
void exibirEstatisticasFinais(Atendimento* a);
```

São cinco funções principais que criam a lógica de atendimento e a coleta de estatísticas do simulador.

4.2. atendimento.c

- Atendimento* inicializarAtendimento();
 - Aloca memória para a estrutura que armazena todas as estatísticas (contadores de clientes, itens, tempo) e inicializa a fila que guardará o histórico de clientes já atendidos.
- void liberarAtendimento(Atendimento* a);
 - Libera os recursos alocados para a fila de histórico e para a própria estrutura de estatísticas.
- void atenderProximoCliente(Fila* fComum, Fila* fPreferencial, Atendimento* a);
 - Implementa a regra de negócio (atender 2 clientes comuns para 1 preferencial), decide qual cliente será o próximo, o remove da fila de espera correspondente, atualiza todas as estatísticas e o adiciona na fila de histórico.
- void exibirHistoricoAtendimentos(Atendimento* a);
 - Exibe a lista de clientes armazenados na fila de histórico.
- void exibirEstatisticasFinais(Atendimento* a);
 - Apresenta o resumo do desempenho do sistema. Com base nos dados acumulados na estrutura Atendimento, ela calcula e exibe o total de atendimentos, o total de itens processados e o tempo médio de atendimento para cada tipo de fila.

4.3. Dificuldades

As maiores dificuldades na função atenderProximoCliente foram aplicar corretamente a regra de atendimento (2 clientes comuns para 1 preferencial), controlar o estado da alternância entre as filas e evitar erros ao lidar com filas vazias. Também foi necessário garantir que todas as estatísticas fossem atualizadas após cada atendimento e que o cliente fosse movido corretamente para o histórico, exigindo atenção no uso de ponteiros e na integração com as demais estruturas do sistema.

5. Main

5.1. main.c

- void limparBuffer();
 - Função criada no main.c para limpar todo e qualquer buffer do sistema - como espaço ou \n
- void popularFilasTeste(Fila *comum, Fila* pref);
 - Função criada no main.c apenas para agilizar o processo de cadastrar 3 clientes na fila comum e 2 clientes na fila preferencial.
 - Utiliza a função enfileirar e a função criarCliente para criar dados genéricos - apenas de teste
- Fila filaComum, filaPreferencial;
 - Inicializa as filas de clientes - Comum e Preferencial
 - Utiliza as funções de fila.c
- Atendimento* a = inicializarAtendimento();
 - Inicializa o atendimento
 - Utiliza as funções de atendimento.c
- Loop do-while
 - Todos os cases (opções) do sistema
 - Adicionar cliente Comum
 - Adicionar cliente Preferencial
 - Atender clientes na fila
 - Exibir lista
 - Exibir Histórico
 - Exibir estatísticas
 - Testar funcionalidades de cliente.c
 - Gerar clientes genéricos rapidamente
 - Sair
- Case 1
 - Criar novo cliente na fila COMUM
 - Utiliza funções de cliente.c
 - Cliente novocliente = criarCliente
 - Utiliza funções de fila.c
 - Enfileirar(&filaComum, novoCliente)
 - Armazena a quantidade de itens - &itens
- Case 2
 - Criar novo cliente na fila PREFERENCIAL
 - Utiliza funções de cliente.c
 - Cliente novocliente = criarCliente
 - Utiliza funções de fila.c

- Enfileirar(&filaPreferencial, novoCliente)
 - Armazena a quantidade de itens - &itens
- Case 3
 - Chama a função atenderProximoCliente(&filaComum, &filaPreferencial, a)
 - Utiliza a lógica necessária para atender o pedido de 2x1 - 2 comuns, 1 preferencial.
- Case 4
 - Chama a função atenderProximoCliente(&filaComum, &filaPreferencial, a)
 - Utiliza a lógica necessária para atender o pedido de 2x1 - 2 comuns, 1 preferencial.
- Case 5
 - Chama a função exibirHistoricoAtendimentos(a);
 - Utiliza a lógica implementada em atendimento.c
- Case 6
 - Chama a função exibirEstatisticasFinais(a);
 - Utiliza a lógica implementada em atendimento.c
- Case 7
 - Utiliza as funções implementadas em cliente.c para testar as funcionalidades e fórmulas
 - Função Cliente teste = criarCliente
 - Função imprimirCliente(&teste)
 - faz a verificação - comum ou presencial
 - Calcula o tempo conforme função e fórmula implementada em cliente.c - tempoCalculado = calcularTempoAtendimento(teste.itens)
- Case 8
 - popularFilasTeste(&filaComum, &filaPreferencial);
 - Função auxiliar criada diretamente no main.c para agilizar o processo
- Case 0
 - Chama as funções de liberar memória em fila.c e atendimento.c

5.2. Dificuldades

As maiores dificuldades no desenvolvimento do main.c foram o controle de entrada de dados, onde foi necessário implementar uma limpeza de buffer para corrigir possíveis erros de leitura entre números e textos, por exemplo. Também, teve o desafio na parte de ponteiros, porque precisei garantir a passagem correta de referências para as funções alternarem nas filas originais e não cópias.

6. Conclusão

Conclui-se que o projeto do Sistema de Gerenciamento de Filas de Supermercado permitiu a aplicação prática dos conceitos aprendidos na disciplina de Estrutura de Dados 1, especialmente em relação ao uso de Filas e à modularização de códigos.

A escolha de filas baseadas em listas encadeadas para organizar os clientes (Comuns e Preferenciais) mostrou-se a solução adequada para respeitar a ordem de chegada e as regras de prioridade do sistema.

Apesar das dificuldades encontradas na manipulação de ponteiros, no tratamento de entrada de dados (buffer) e na integração entre os arquivos .c e .h, o trabalho foi fundamental para consolidar o entendimento sobre a alocação dinâmica de memória e a importância de criar códigos organizados e eficientes em linguagem C.